

گزارش پروژه چت Client/Server

۱ مقدمه

این پروژه یک برنامه چت چند کاربره با معماری Client/Server است که با Java پیاده سازی شده. سیستم از Socket TCP برای ارتباط استفاده می‌کند و شامل بخش‌های کلاینت، سرور و مازول مشترک است.

۲ پیاده سازی سرفصل‌های HW۳

۱.۲ Generics

در پروژه از Generics به خوبی استفاده شده:
Packet<T>: کلاس اصلی پروتکل ارتباطی که جنریک است و می‌تواند هر نوع Payload را نگه دارد:

این کلاس در همه جا استفاده شده مثلاً، `Packet<ChatMsgReq>`، `Packet<ErrorPayload>` و غیره. این کار safety type رو بهتر می‌کنه و نیاز به cast کردن رو کم می‌کنه.

۲.۲ Handling Exception

۱.۲.۲ Exceptions Custom

چند Exception سفارشی تعریف شده:

• **DuplicateUsernameException**: وقتی نام کاربری تکراری باشه

- **RoomNotFoundException**: وقتی روم پیدا نشه
- **FileTransferException**: برای خطاهای مربوط به فایل
- **InvalidCommandException**: برای دستورات نامعتبر

۲.۲.۲ ExceptionManager

یک کلاس مرکزی به اسم ExceptionManager تعریف شده که همه Exceptionها رو به <ErrorPayload> Packet تبدیل می‌کنه. این کار باعث می‌شه خطاها به صورت استاندارد به کلاینت ارسال بشن:

۳.۲.۲ مقایسه روش قدیمی و Exception

روش قدیمی (مثلا return: code)

• مزایا: سریع‌تر، کنترل flow ساده‌تر

• معایب: فراموش کردن چک کردن کد برگشتی، کدهای نامرتب، مشکل در توابع تو در تو

روش Exception:

• مزایا: کد تمیزتر، اجباری بودن هندل کردن خطا، انتقال خطا به لایه بالا راحت‌تر، جداسازی منطق عادی از خطا

• معایب: کمی کندتر، ممکنه فراموش بشه catch بشه

در این پروژه از Exception استفاده شده چون برای شبکه و I/O مناسب‌تره و کد خواناتر می‌شه.

۴.۲.۲ finally و Exception Runtime

RuntimeException: بعضی از های Exception سفارشی مثل InvalidComman- dException از RuntimeException ارث بری می‌کنن چون خطاهای منطقی هستن و نیازی به declare کردن نیست.

finally: در چند جا از finally استفاده شده:

۱. **RoomState**: برای باز کردن lock حتما از finally استفاده شده تا حتی اگر exception بیفته lock باز بشه:

۲. **ClientHandler**: برای cleanup کردن اتصال و session حتما از finally استفاده شده:

این کار باعث می‌شود منابع حتما آزاد بشن حتی اگر exception بیفته.

۳.۲ Section Critical و Concurrency

۱.۳.۲ Sectionها Critical

۱. **مدیریت اعضای روم**: اضافه/حذف اعضا از RoomState باید thread-safe باشه چون چند thread همزمان ممکنه به یک روم دسترسی داشته باشن. از Reentrant-Lock استفاده شده:

۲. **اضافه کردن پیام به history**: وقتی پیام جدید میاد باید به history روم اضافه بشه و این کار باید thread-safe باشه. از همون lock روم استفاده شده.

۳. **ارسال Packet از ObjectOutputStream**: thread-safe ObjectOutputStream نیست پس برای ارسال از synchronized استفاده شده. در: UserSession.send:

و در ClientHandler هم وقتی session نداریم:

۴. Broadcast به اعضای روم: وقتی می‌خواهیم پیام رو به همه اعضای روم بفرستیم، اول snapshot از لیست اعضا می‌گیریم تا اگر در حین broadcast عضو اضافه یا حذف بشه مشکل پیش نیاد.

۲.۳.۲ ساختارهای Thread-Safe

از ConcurrentHashMap برای نگه داری کاربران آنلاین، روم‌ها و فایل‌ها استفاده شده. این کار دسترسی به این Map‌ها رو thread-safe می‌کنه بدون نیاز به lock اضافی.

۴.۲ Serialization و Network

ارتباط از طریق Socket TCP برقرار می‌شه. از ObjectOutputStream و ObjectInputStream Stream برای ارسال/دریافت object استفاده شده. ترتیب ساخت stream‌ها مهمه تا deadlock پیش نیاد (اول OutputStream ساخته می‌شه).

۵.۲ Pattern Command

هر دستور یک کلاس جداگانه داره که interface Command رو implement می‌کنه. این کار باعث می‌شه کد تمیزتر و قابل توسعه‌تر بشه. مثلاً، Upload- JoinCommand و Command و غیره.

۳ نتیجه‌گیری

این پروژه با استفاده از مفاهیم Concurrency Handling، Exception Generics و Design Pattern‌ها پیاده سازی شده. استفاده از ReentrantLock و Concurrent-HashMap باعث thread-safety شده و استفاده از Exception Custom و Exception tionManager مدیریت خطا رو بهتر کرده.